

SECURE CODING REVIEW REPORT

CodeAlpha Cyber Security Internship

Task 3 – Secure Coding Review

Submitted By: Kashaf Waheed

Internship: CodeAlpha Cyber Security Internship

Technologies Used: Python 3 · Visual Studio Code · getpass Module

Submission Date: July 2026

Table of Contents

1. Project Overview.....	3
2. Project Objectives.....	3
3. Vulnerable Login System.....	3
4. Security Vulnerabilities Identified.....	4
5. Secure Coding Improvements.....	4
6. Comparison: Vulnerable vs Secure System.....	5
7. Testing Results.....	5
8. Screenshots.....	6
9. Future Improvements.....	8
10. Conclusion.....	8
11. Learning Outcomes.....	9
Appendix A: Project Folder Structure.....	10

1. Project Overview

Secure coding is the practice of writing software in a way that guards against the accidental introduction of security vulnerabilities. Rather than treating security as an afterthought, secure coding embeds protective measures — such as input validation, credential protection, and access control — directly into the development process.

Secure coding is important because even a single overlooked flaw, such as a hardcoded password or an unlimited login attempt, can be exploited by attackers to gain unauthorized access, steal sensitive data, or compromise an entire system. As software increasingly manages financial transactions, personal data, and critical infrastructure, the cost of insecure code has grown far beyond inconvenience.

The purpose of this project is to demonstrate the practical difference between insecure and secure implementations of a simple authentication system. A basic Python login program was first built with intentional weaknesses, then rebuilt using secure coding principles to address each identified flaw.

This project demonstrates the ability to identify common security vulnerabilities in source code, apply industry-recognized remediation techniques, and document the entire process in a clear, professional manner suitable for a cybersecurity internship deliverable.

2. Project Objectives

- Understand secure coding principles and their role in software security.
- Identify vulnerabilities present in an insecure authentication program.
- Develop a secure login system that addresses the identified weaknesses.
- Implement basic security mechanisms such as password masking, input validation, and login-attempt limiting.
- Compare the vulnerable and secure implementations to evaluate the effectiveness of the improvements.

3. Vulnerable Login System

The original program, `vulnerable_login.py`, is a simple console-based authentication script built to demonstrate common security mistakes found in real-world code. It intentionally omits standard protections so that each weakness can be clearly identified and later remediated in the secure version.

Features

- Username and password based login.
- Simple authentication logic using direct string comparison.
- Basic console (terminal) interface with no graphical elements.

Weaknesses

The program intentionally contains multiple vulnerabilities for educational purposes. These weaknesses were introduced deliberately to provide a realistic baseline for identifying security flaws, and are analyzed in detail in the following section.

4. Security Vulnerabilities Identified

The table below summarizes the vulnerabilities discovered during the review of the original login program, along with the associated risk level.

No.	Vulnerability	Description	Risk
1	Hardcoded Credentials	Username and password stored directly in source code	High
2	Weak Password	Password is too simple and easily guessable	High
3	Password Visible	Password is displayed in plain text while typing	Medium
4	Unlimited Login Attempts	No restriction on retries, allowing brute-force attacks	High
5	No Input Validation	Program accepts empty username or password input	Medium

5. Secure Coding Improvements

To address the vulnerabilities identified above, the program was rebuilt as `secure_login.py` with the following improvements.

Improvement	Description
Password Masking	Implemented using Python's <code>getpass</code> module so input is never shown on screen
Login Attempt Limit	Maximum of three login attempts per session

Improvement	Description
Brute Force Protection	Account locks automatically after repeated failed attempts
Input Validation	Prevents empty username or password from being submitted
Improved Program Flow	Used loops and a break statement for cleaner, more predictable control flow

6. Comparison: Vulnerable vs Secure System

The table below provides a side-by-side comparison highlighting how each security concern was addressed between the two versions of the program.

Feature	Vulnerable	Secure
Password Visible	Yes	No
Hardcoded Credentials	Yes	Yes (Educational Example)
Login Attempts	Unlimited	Limited to 3
Input Validation	No	Yes
Brute Force Protection	No	Yes

7. Testing Results

The secure version of the program was tested against the following scenarios to confirm that each security improvement functions as intended.

Test Case	Expected Result	Status
Correct Login	Login Successful	Passed
Incorrect Password	Error Message	Passed
Empty Input	Validation Message	Passed
Three Wrong Attempts	Account Locked	Passed

8. Screenshots

The figures below illustrate the secure login system in operation. Screenshot placeholders are provided and can be replaced with actual captures.

```
=====
Secure Login System
=====
Enter Username: admin
Enter Password:

Login Successful!
Welcome, admin
Access Granted
Loading Dashboard...
```

Figure 1: Successful Login

```
=====
Secure Login System
=====
Enter Username: admin
Enter Password:

Invalid Username or Password!
Remaining Attempts: 2
```

Figure 2: Incorrect Password

```
=====
Secure Login System
=====
Enter Username:
Enter Password:

Username and Password cannot be empty.
Enter Username: █
```

Figure 3: *Empty Input Validation*

```
Enter Username: aa
Enter Password:

Invalid Username or Password!
Remaining Attempts: 1
Enter Username: fu
Enter Password:

Invalid Username or Password!
Remaining Attempts: 0

Account Locked!
Please try again later.
```

Figure 4: *Account Locked*

```

secure_login.py > ...
1  import getpass
2
3  print("=" * 40)
4  print("      Secure Login System")
5  print("=" * 40)
6
7  attempts = 3
8
9  while attempts > 0:
10     username = input("Enter Username: ")
11     password = getpass.getpass("Enter Password: ")
12
13     # Input Validation
14     if username.strip() == "" or password.strip() == "":
15         print("\nUsername and Password cannot be empty.")
16         continue
17
18     if username == "admin" and password == "123456":
19         print("\nLogin Successful!")
20         print("Welcome,", username)
21         print("Access Granted")
22         print("Loading Dashboard...")
23         break
24
25     else:
26         attempts -= 1
27         print("\nInvalid Username or Password!")
28         print("Remaining Attempts:", attempts)
29
30 if attempts == 0:
31     print("\nAccount Locked!")
32     print("Please try again later.")

```

Figure 5: Source Code

9. Future Improvements

- Store user credentials in a secure database rather than in code.
- Implement password hashing (e.g., bcrypt) instead of plain-text comparison.
- Add Two-Factor Authentication (2FA) for stronger identity verification.
- Implement CAPTCHA to further deter automated brute-force attempts.
- Maintain login logs for auditing and anomaly detection.
- Improve password policy with minimum length and complexity requirements.

10. Conclusion

This project successfully demonstrated the process of identifying and remediating common security vulnerabilities in a simple authentication system. The initial review of the vulnerable login program revealed several high and medium risk issues, including hardcoded credentials,

an unmasked password field, unlimited login attempts, and a lack of input validation. Each of these weaknesses was addressed in the secure version through targeted coding practices such as password masking with the `getpass` module, a strict login-attempt limit with automatic account locking, and validation to reject empty input. The resulting comparison and testing results confirm that the secure implementation meaningfully reduces the system's exposure to brute-force attacks and credential exposure. Overall, this exercise reinforced the importance of building security into software from the earliest stages of development rather than treating it as an afterthought.

11. Learning Outcomes

During this project, I learned:

- Secure Coding Fundamentals
- Risks of Hardcoded Credentials
- Input Validation techniques
- Password Protection methods
- Brute Force Attack Prevention
- Authentication Security best practices
- Python Security Practices
- Secure Software Development Basics

Appendix A: Project Folder Structure

Task-3-Secure-Coding-Review/

- ├── vulnerable_login.py
- ├── secure_login.py
- ├── README.md
- ├── Secure_Coding_Review_Report.pdf
- └── screenshots/